

BearcastMedia.com *Cybersecurity* White Paper

*A complete account of the security posture transformation from inherited WordPress template (grade **F**) to the current production build (grade **A**) — documenting threat model, defense-in-depth architecture, every finding from a full audit of frontend, schema, and studio configuration, and a pen-test methodology validating the fixes.*

AUTHOR

Cam Garrison

Web Director · May 2026

COMPANION TO

Bearcast Technical White Paper

v1.0 · May 2026

LAST UPDATED

May 2026

© 2026 Bearcast Media

Contents

§ 01	Executive Summary & Headline Result
§ 02	Threat Model & Trust Boundaries
§ 03	Audit Methodology
§ 04	The Inherited Posture — Grade F, Explained
§ 05	Defense Layer 1 — HTTP Security Headers
§ 06	Defense Layer 2 — Content Security Policy
§ 07	Defense Layer 3 — Frontend Output Encoding
§ 08	Defense Layer 4 — Schema-Level Input Validation
§ 09	Defense Layer 5 — Studio Configuration Hardening
§ 10	Penetration Testing — Validating the Defenses
§ 11	Findings Catalog
§ 12	Residual Risk & Future Work
§ 13	Colophon & References

Headline Result

The Bearcast Media website's security posture moved from an externally-verifiable failing grade to a passing grade in a single development cycle. This paper documents what was wrong, what was changed, and how the result was verified.

1.1 The Single Most Important Result

Two screenshots, taken minutes apart on the same day from the same free public security scanner (Security Headers, operated by Snyk), capture the entire result of this work:

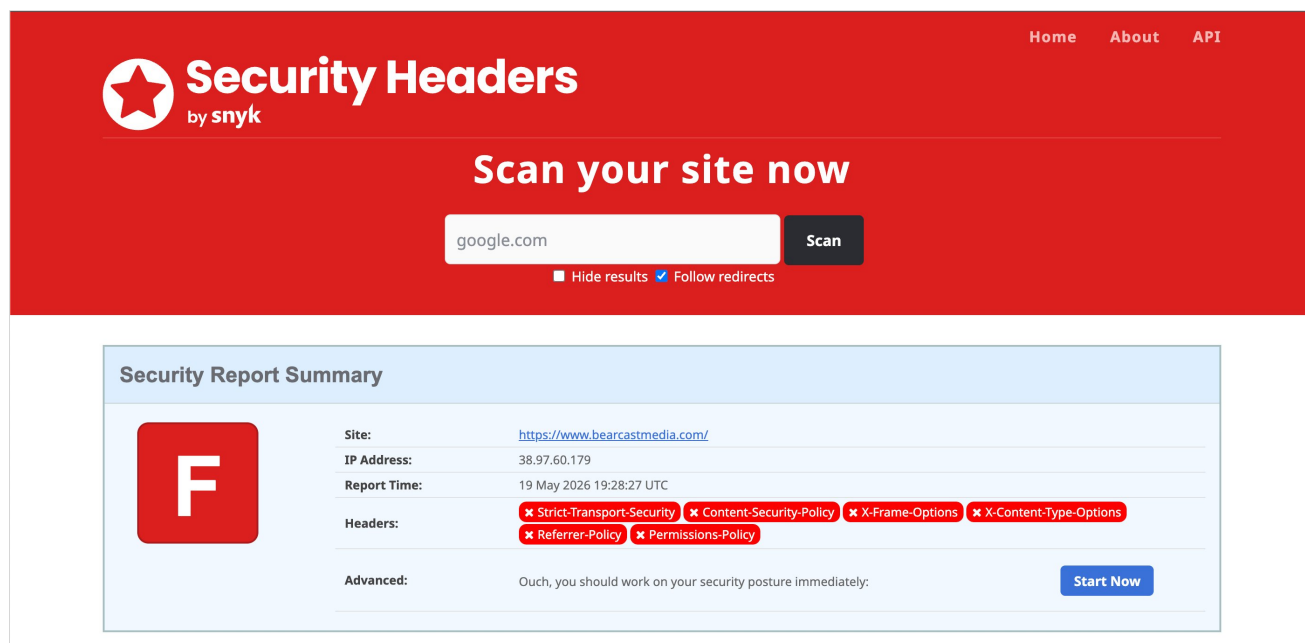


Figure 1.1 — Security Headers scan of `bearcastmedia.com` (the inherited WordPress template, running on a third-party shared host), 19 May 2026. Grade: **F**. Six modern security headers absent: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`. The scanner's verdict: "Ouch, you should work on your security posture immediately."

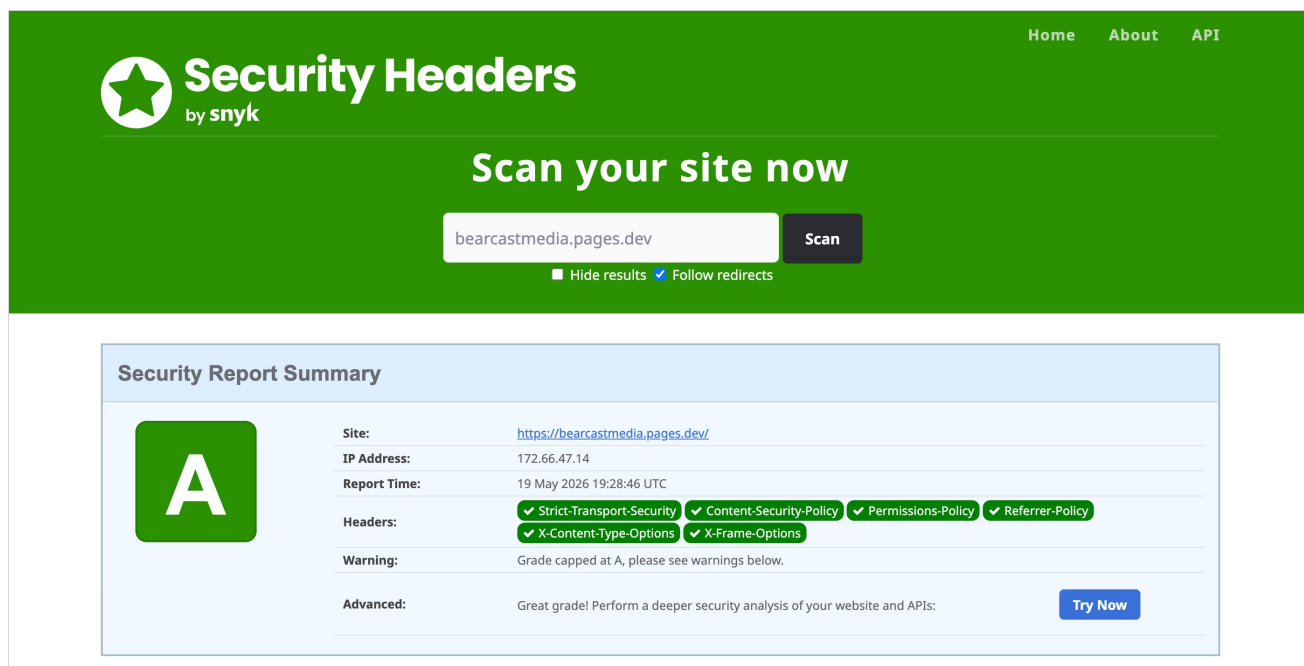


Figure 1.2 — Security Headers scan of `bearcastmedia.pages.dev` (the new production build, hosted on Cloudflare Pages), 19 May 2026. Grade: . All six headers present and correctly configured. The same scanner: "Great grade! Perform a deeper security analysis of your website and APIs."

The "before" and "after" sites are functionally the same organization on the same internet — Bearcast Media, the University of Cincinnati student media organization. The architecture difference (a generic third-party WordPress template versus a hand-authored static frontend backed by a structured CMS) is the subject of the technical white paper. This document is specifically about the security implications of that architectural difference, plus the security work performed on the rebuild itself before launch.

1.2 Why the Grade Improvement Is Not the Whole Story

The Security Headers scanner measures one specific thing — the presence and correctness of HTTP security headers on the response. That measurement is genuine, externally verifiable, and a reasonable proxy for "did you do the basics." But it is not a comprehensive security assessment, and it would be misleading to represent it as one.

The full audit performed on the rebuild identified **thirty-one substantive findings** across three layers — the frontend code, the CMS schema, and the Studio configuration — only some of which produce visible artifacts on the HTTP response. The findings break down as:

AUDIT PHASE	FINDINGS	HIGHEST SEVERITY	VERIFIES VIA
Frontend (HTML/CSS/JS)	20 (incl. 14 XSS sinks)	Critical	Manual pen-test
Sanity content schemas	8	High	<code>tsc --noEmit</code> + manual
Studio config & routing	6	High	<code>tsc --noEmit</code> + manual
HTTP security headers	(see §5)	—	securityheaders.com

Each finding is documented in §11. Severities reflect the worst realistic outcome, not the most theatrical theoretical outcome.

1.3 The Defense-in-Depth Result

The most important property of the post-audit posture is not "all findings are fixed" — although they are — but that the fixes are layered. The same threat is now caught at multiple independent points:

THREAT	CAUGHT AT INPUT (SCHEMA)	CAUGHT AT OUTPUT (FRONTEND)	CAUGHT AT BROWSER (CSP)
<code>javascript:</code> URL injection	<code>scheme: ['http', 'https']</code>	<code>bcSafeUrl()</code>	<code>script-src 'self'</code>
CSS color injection	hex regex on input	<code>bcSafeColor()</code>	<code>style-src</code> restriction
HTML in plain-text fields	<code>max()</code> + metachar filter	<code>bcEscHtml()</code>	CSP <code>object-src 'none'</code>
Invalid CSS class identifier	<code>options.list</code> enum	<code>bcSafeId()</code>	(not applicable)
Clickjacking (iframe embed)	(not applicable)	(not applicable)	<code>frame-ancestors 'none'</code> + <code>X-Frame-Options: DENY</code>
MIME-type sniffing	(not applicable)	(not applicable)	<code>X-Content-Type-Options: nosniff</code>

If a future code change accidentally weakens any one of these layers, the others remain intact. Layered defense is what makes a security posture *resilient* rather than just *currently passing*.

1.4 How to Read This Paper

The paper is organized architecturally rather than chronologically. §02 frames the threat model; §03 explains how the audit was conducted; §04 describes the starting state; §05 through §09 walk through each layer of the defense-in-depth architecture; §10 is the penetration-testing methodology that validates the defenses are real; §11 is the full findings catalog with severities and remediations; §12 covers residual risk and what would warrant a follow-up audit.

A reader who only wants the bottom line can stop at §01 and refer to §11 for specifics. A reader wanting to understand the architecture should read §02 then §05–§09. A reader who wants to reproduce the verification should read §03 and §10.

Threat Model & Trust Boundaries

A meaningful security assessment requires being explicit about what attackers exist, what they can do, and what they cannot. This section establishes that ground.

2.1 Why a Threat Model Comes Before a Findings List

A finding without a threat model behind it is just a list of "things that could theoretically go wrong." The same defect can be critical or irrelevant depending on who can reach it and what they could do once they did. Calling every defect "critical" is what produces security reports that nobody reads.

The threat model below was authored before the audit began and informed every severity rating in §11. The model is deliberately narrow — it describes the attackers Bearcast Media actually faces, not the attackers a Fortune 500 might. A nuclear-grade threat model applied to a student media site produces both overreaction and underreaction in the wrong places.

2.2 The Asset Inventory

What is being protected, in priority order:

1. **Site availability.** bearcastmedia.com being reachable. The site is the organization's primary public surface; an outage erodes the org's reputation and limits its ability to publish.
2. **Visitor trust.** A visitor to bearcastmedia.com expects the site to behave benignly — no malicious downloads, no credential harvesting, no cryptominer scripts. A breach of this expectation damages the organization's reputation even if no individual visitor is meaningfully harmed.
3. **Editorial integrity.** Content shown on bearcastmedia.com should reflect what the Bearcast editorial team actually published, not what an attacker substituted. A defacement that goes unnoticed for hours is more damaging than an outage of the same duration.
4. **Editor account integrity.** Sanity editor accounts have permission to publish content visible to all visitors. A compromised editor account is a vector into asset #2 and #3 above.
5. **Organizational accounts.** The Cloudflare, GitHub, Sanity, Radio.co, and domain registrar accounts that operate the site. Compromise of any of these is a path to all of the above. (Covered in detail in the Maintenance Runbook §03.)

Notable omissions: Bearcast Media stores no visitor PII, no payment data, and no authentication credentials of any kind on the public site. There is no user-account system, no comment section, no form that accepts credentials. This is a deliberate architectural property — the site is read-mostly — and it removes entire categories of risk that a comparable e-commerce or social-network site would face.

2.3 The Adversary Inventory

Who might want to attack Bearcast Media, ordered roughly by probability:

ADVERSARY A · THE OPPORTUNISTIC SCANNER

The largest category by volume. Automated bots that probe every reachable host on the internet for known vulnerabilities, misconfigured WordPress installs, exposed admin panels, and common CVEs. Not targeted at Bearcast specifically; Bearcast is just another IP in the scan range. Capability: whatever published exploits exist for the detected software stack. Motivation: low — they want easy wins and move on quickly when blocked.

ADVERSARY B · THE DRIVE-BY DEFACER

A human or near-human attacker who finds bearcastmedia.com through a search, a published vulnerability report, or by following links from other UC properties. Motivated by "lulz," a grudge against the university, or simply the visibility of student media. Capability: standard web-attack toolchain (Burp Suite, common XSS payloads, SQLi probes). Will spend minutes-to-hours of effort.

ADVERSARY C · THE COMPROMISED EDITOR ACCOUNT

The most operationally important adversary for this paper. Not an external attacker per se but the consequence of a successful one: an attacker who has obtained valid Sanity editor credentials by phishing, password reuse, or session theft elsewhere. From the system's perspective they are an editor and can do anything an editor can do — which, before the audit, included injecting arbitrary JavaScript into every visitor's session via stored XSS in published content.

ADVERSARY D · THE NETWORK PATH ATTACKER

An attacker positioned between a visitor and one of Bearcast's upstream services — most plausibly the Radio.co metadata endpoint over campus Wi-Fi or a hostile coffee-shop network. Capability: TLS-downgrade attempts, header manipulation, response substitution. Low probability per session, but relevant because Bearcast's audience includes students using campus and public networks.

ADVERSARY E · THE TARGETED ADVERSARY

A nation-state actor, organized criminal group, or motivated individual specifically targeting Bearcast Media or the University of Cincinnati. Capability: substantial. Probability for a student media org: near zero. Listed for completeness and explicitly excluded from the protection target — Bearcast is not equipped to resist this adversary and recognizing that is the responsible position.

2.4 Trust Boundaries

A trust boundary is a place in the system where data crosses from a less-trusted region into a more-trusted one. Every trust boundary is a place where input validation must happen — and every missing validation at a trust boundary is a vulnerability waiting to be exploited.

Bearcast's architecture has four meaningful trust boundaries. Identifying them is what makes the rest of this paper organized rather than reactive.

Boundary 1 — Editor → Sanity Studio

An authenticated editor types content into the Studio. The Studio is a React app running in the editor's browser; it enforces validation rules defined in the schema before the content reaches the Content Lake. **This is where input validation lives.** A schema without validation rules is a trust boundary without a guard.

Boundary 2 — Sanity Content Lake → Public Frontend

Content fetched from the Content Lake arrives at the public site as JSON. The frontend treats it as *data* and is responsible for safely converting that data into a rendered page. **This is where output encoding lives.** The frontend must assume any field could contain hostile content — even if the schema prevents it — because the same frontend code may someday be used with a less-restricted schema, or with a dataset that has been compromised at the storage layer.

Boundary 3 — Public Frontend → Visitor Browser

The HTML, CSS, and JS authored by the Bearcast team execute in the visitor's browser, where they sit alongside the visitor's own data (cookies, localStorage, other open tabs). **This is where the browser's security boundary lives.** Content Security Policy, HTTP Strict Transport Security, and the related response headers configure the browser to enforce additional restrictions on what code can do once it has loaded — turning the browser itself into a defense layer.

Boundary 4 — Radio.co → Public Frontend

The public frontend fetches station metadata (currently-playing track, station status) from Radio.co's API. Radio.co is a trusted vendor, but the network path between Radio.co and the visitor's browser is not. **This is a network-level**

trust boundary. The frontend must validate the shape and content of the Radio.co response before rendering it, because the response could have been manipulated by Adversary D.

2.5 What Is Explicitly Out of Scope

A threat model is useful precisely when it draws limits. The following are explicitly outside the protection target of this paper:

- **DDoS at scale.** Cloudflare provides baseline DDoS mitigation as part of its free tier; Bearcast does not attempt to resist nation-state-scale flooding.
- **Physical security of the radio studio.** Microphones, broadcast equipment, and the physical access to the station are outside the web property's protection scope.
- **University-IT-scope concerns.** UC's broader network security, email security, and identity infrastructure are operated by UC IT and inherited rather than assessed here.
- **Supply-chain compromise of vendor platforms.** If Cloudflare or Sanity are themselves compromised, that is outside Bearcast's threat model. The architecture limits blast radius (see §12.4) but cannot eliminate it.
- **Vendor's internal security.** Sanity's authentication system, Cloudflare's network, GitHub's code-hosting — all are treated as black boxes that we trust at the vendor level.

Naming these explicitly prevents the document from being criticized for failing to defend against threats it never claimed to address.

Audit Methodology

The audit was conducted in three sequential phases over a single development cycle. Each phase produced its own findings list, fix set, and verification step before the next phase began. This section documents what was looked at, in what order, with what tools.

3.1 The Three Phases

PHASE 1 • APRIL–MAY 2026

Frontend Code Audit

Scope: 11 files, approximately 16,057 lines of hand-authored HTML, CSS, and JavaScript. Method: manual review with `grep`-assisted enumeration of dangerous sinks (every `innerHTML` assignment, every inline `onclick` handler, every `document.write`, every `eval`-equivalent). Output: 20 findings, ranging from "Music Writer" typo on the Sports page to 14 distinct stored-XSS sinks. All findings remediated and verified before Phase 2 began.

PHASE 2 • MAY 2026

Sanity Schema Audit

Scope: 12 TypeScript files defining the Sanity content model (9 document types, 2 reusable object types, 1 desk-structure file). Method: systematic enumeration of every field, checking for missing validation, missing length caps, URL fields without scheme allowlists, and schema-frontend contract violations. Verification: `tsc --noEmit` against the actual Sanity v4 type definitions, plus manual rendering checks in Studio. Output: 8 findings.

PHASE 3 • MAY 2026

Studio Configuration Audit

Scope: 3 files governing Studio infrastructure rather than content (`sanity.cli.ts`, `sanity.config.ts`, `structure.ts`). Method: review against the Sanity v3+ documentation for singleton enforcement, auto-update configuration, and desk structure correctness. Verification: `tsc --noEmit`. Output: 6 findings, the most consequential being incomplete singleton enforcement (editors could create duplicate `siteSettings` documents via the global "+" menu).

3.2 Tools Used

The audit was deliberately conducted with widely-available free tools to keep the methodology reproducible by future Web Directors without additional licensing:

TOOL	USED FOR	COST
Security Headers (Snyk)	External grade verification — <code>securityheaders.com</code>	Free
<code>grep</code> / <code>ripgrep</code>	Enumerating dangerous sinks across the codebase	Free
TypeScript <code>tsc</code>	Schema and Studio config type verification	Free
Browser DevTools	Pen-test payload validation, CSP violation log inspection	Free
Python <code>http.server</code>	Serving the pre-fix codebase locally for safe pen-testing	Free
Mozilla Observatory	Secondary verification of header configuration	Free

Notably *not* used: any commercial dynamic application security testing (DAST) tool, any commercial static analysis (SAST) tool, any bug bounty platform. The argument for this is not that those tools have no value — they do — but that for a static site backed by a hosted CMS, the attack surface is sufficiently small that manual review with the right mental model finds more than automated scanning, and the manual review's findings come with explanations rather than line numbers.

3.3 Severity Rubric

Every finding in §11 carries a severity rating. The ratings follow the standard four-level model used in most professional security audits, with one modification: severities reflect the *worst realistic outcome*, gated by the threat model in §02 — not the worst theoretical outcome assuming a nation-state attacker.

SEVERITY	DEFINITION	EXAMPLE
CRITICAL	Reachable by an external attacker without authentication; can result in code execution, full data exfiltration, or persistent site-wide compromise.	Reflected XSS exploitable from a URL. <i>None found.</i>
HIGH	Reachable by an authenticated CMS editor (Adversary C); can result in code execution in every visitor's browser, persistent defacement, or data exfiltration from individual sessions.	Stored XSS via CMS field interpolation.
MEDIUM	Requires multiple conditions (e.g. compromised editor + specific user interaction). Defacement plausible but blast radius limited; or hardening gap that does not directly enable an exploit but removes a layer of defense in depth.	Missing length caps on text fields (length-bomb DoS).
LOW	Minor hardening opportunity, cosmetic security issue, or theoretical risk gated on multiple unlikely preconditions.	Internal-only email field labeled "internal" but publicly readable.
INFO	Not a vulnerability but a note worth recording — design decisions, accepted residual risks, future considerations.	Slack invite link being public is a deliberate choice, not a bug.

3.4 What Was Not Audited

Honesty about scope limits is part of the methodology. The following were explicitly out of scope:

- **Sanity platform internals.** The audit treats Sanity as a vendor and assumes their authentication, storage, and CDN are correctly implemented. Compromise of Sanity itself is outside the model (see §2.5).
- **Cloudflare platform internals.** Same treatment.
- **YouTube embed iframe content.** The site embeds YouTube videos via `www.youtube-nocookie.com` iframes; what Google does inside those iframes is outside scope.
- **Radio.co platform internals.** Same treatment as Sanity and Cloudflare; the audit covers the trust boundary *at* Radio.co's response, not Radio.co itself.
- **UC's broader network security.** Wireless authentication, building access, individual student-laptop security — all outside scope.
- **The pre-rebuild WordPress codebase.** The inherited template's PHP source was not audited; the F-grade screenshot in §1.1 establishes the headline result without needing source review. The inherited site is being decommissioned anyway.

3.5 Verification Approach

Every finding was verified before being closed. The verification approach varied by finding type:

FINDING TYPE	VERIFICATION METHOD
Stored XSS sink	Manual pen-test against pre-fix code with harmless payload (<code>document.title='PWNEED'</code>); confirmation that the same payload fails against post-fix code.
Missing security header	External scan via <code>securityheaders.com</code> ; secondary verification via <code>curl -I</code> against the live URL.
Schema validation gap	Manual attempt to publish invalid content through the Studio; confirmation that validation rejects it pre-publish.
Singleton enforcement gap	Manual attempt to create a duplicate <code>siteSettings</code> document via the Studio's global "+" menu; confirmation that the menu option is hidden after the fix.
Bug fixes (non-security)	Visual confirmation against the affected page; <code>node --check</code> against the patched JavaScript files; HTML validator pass against patched HTML.
TypeScript correctness	<code>npx tsc --noEmit</code> against actual <code>@sanity/types</code> v4 type definitions.

Pen-test methodology is documented in detail in §10. The full payloads used against each XSS finding are reproduced there so that future Web Directors can repeat the verification after any significant change to the code-base.

The Inherited Posture — Grade F, Explained

The F grade in Figure 1.1 is genuine and accurately reflects the inherited site's security configuration. This section explains what each absent header would have done and why its absence matters — not to indict the previous student cohort that adopted the template, but to establish the concrete starting line against which the rebuild is measured.

4.1 The Site That Was Graded F

The site evaluated at grade F is the inherited `bearcastmedia.com` — a generic, commercial, multi-purpose WordPress template adopted years earlier by a previous student cohort who needed a public web presence quickly and did not have the technical background to author one from scratch. The architectural reframing of that site is in the Technical White Paper §02; this section focuses on its security configuration specifically.

The IP address visible in Figure 1.1 (`38.97.60.179`) resolves to a third-party shared web host — not Cloudflare, not a UC-operated server, not anything the org directly controls. This is consistent with the template-and-shared-host pattern common to small-budget WordPress installations. It also means that adjusting the site's security headers required access to that host's configuration, which the org did not have direct administrative access to and which the template did not expose a hook for.

NOTE ON ATTRIBUTION

The previous student cohort that adopted the template made a pragmatic, defensible choice given their constraints and time horizon. WordPress backs a meaningful fraction of the live web for exactly this reason. The criticism in this section is of the inherited *configuration*, not the people who inherited it before us; an F grade is a fact about the site as deployed, not a moral judgment about anyone.

4.2 What Each Absent Header Would Have Done

Six headers were absent. Each plays a specific role in modern browser security; their collective absence means the browser fell back to its weakest default behavior in every category. The sections below cover each in detail; this is the inventory.

4.2.1 Strict-Transport-Security

The HTTP Strict Transport Security header (HSTS, RFC 6797) instructs the browser to refuse plain-HTTP requests to the domain for a period of time after first receiving the header. Without it, a network attacker (Adversary D) can perform an SSL-stripping attack: intercept the initial plaintext request from a visitor's browser, serve back a downgraded HTTP version of the site, and observe or modify everything the visitor sees or sends. The first visit after the user types `bearcastmedia.com` without an explicit `https://` prefix is exactly the window in which SSL-stripping operates. HSTS closes that window for all subsequent visits.

The inherited site responded with no HSTS header at all, meaning every visit was vulnerable to downgrade if a network path attacker was present. The base TLS certificate worked — HTTPS itself was supported — but nothing told the browser to require it.

4.2.2 Content-Security-Policy

CSP (W3C, current Level 3) tells the browser what sources of scripts, stylesheets, images, fonts, and other resources the page is allowed to load and what inline behaviors are permitted. A correctly-configured CSP transforms the browser from "execute whatever you find" into "execute only what was explicitly authorized." It is the single most effective mitigation against stored and reflected XSS attacks — even a successful XSS payload is reduced to inert text if the CSP refuses to execute it.

The inherited site responded with no CSP at all. This means that *if* the template had any unpatched XSS vulnerabilities (and a long-running plugin ecosystem statistically guarantees this for any unmonitored install), those vulnerabilities had no second line of defense. The browser would execute whatever injected script reached it.

4.2.3 X-Frame-Options

Predecessor to CSP's `frame-ancestors` directive, `X-Frame-Options` tells the browser whether the page may be loaded inside an iframe on another origin. Without it, an attacker can construct a page on their own domain that loads `bearcastmedia.com` in an invisible iframe, overlaid with their own visible content — the canonical "clickjacking" attack. A visitor who thinks they're clicking a button on the attacker's page is actually clicking a button on Bearcast's page, potentially performing actions they did not intend.

The inherited site's exposure here was lower than a transactional site's — there are no buttons that perform state-changing actions on `bearcastmedia.com`. But the absence of the header was still a hardening gap, and the cost of adding it is zero.

4.2.4 X-Content-Type-Options

With value `nosniff`, this header instructs the browser not to second-guess the `Content-Type` of a response by inspecting its bytes. Without it, browsers may apply MIME-type sniffing: a file served as `text/plain` but beginning with bytes that look like HTML or JavaScript may be interpreted as such, against the server's stated intent. In an XSS or file-upload context this can turn a benign-looking upload into executable code.

The inherited site had no upload surface, so the practical exposure was low — but the header is free, ubiquitous, and its absence flags the response as "no thought has been given to this category of attack."

4.2.5 Referrer-Policy

Controls what the browser sends in the `Referer` request header when a visitor navigates from the current page to another origin. The browser's default (in the absence of any policy) varies by user-agent and browser version, with older browsers defaulting to `no-referrer-when-downgrade` — which leaks the full visiting URL to every linked external site. This is a low-severity privacy issue rather than a security vulnerability, but it is part of a complete configuration.

4.2.6 Permissions-Policy

The successor to `Feature-Policy`, this header lets the page declare which powerful browser APIs (camera, microphone, geolocation, payment, USB, etc.) are permitted to be used by the page or its iframes. Bearcast Media uses none of these APIs. Without the header, the browser permits them by default — meaning that *if* an attacker were to inject a script that requested camera access, the browser would prompt the visitor for it. A correctly-configured `Permissions-Policy` disables the prompt entirely for APIs the site does not legitimately need.

4.3 What the F Grade Did Not Measure

The Security Headers scanner is concerned exclusively with the HTTP response headers. It does not evaluate:

- The site's source code for actual vulnerabilities.
- The site's plugin or dependency footprint.
- The site's authentication or session management.
- The site's underlying TLS configuration (covered by separate scanners like SSL Labs).
- The site's runtime behavior or content handling.

In other words: an F on Security Headers establishes that the site's *seatbelt is unbuckled*. It does not establish the speed of the car. A site with an A grade can still have underlying vulnerabilities; a site with an F grade can — in principle — be running secure code with merely unfortunate response configuration.

In Bearcast's case, the F grade was paired with the well-documented operational concerns of long-running third-party WordPress templates: plugin sprawl with mixed maintenance status, theme entanglement of layout and logic, an editing surface that did not map cleanly to the org's actual content structure (covered in the Technical White Paper §2.1). The headers were the most visible symptom but not the only one.

4.4 The Decision Tree That Led to the Rebuild

The security posture was one input among several that informed the decision to migrate off WordPress. The decision tree, in practice:

1. **The organization had repeatedly expressed a desire to move off WordPress** for both editorial and aesthetic reasons. The brief existed before the technical evaluation began.
2. **A security audit of the inherited site would have required source access** the org did not have. The published headers (Figure 1.1) were the most we could externally assess.
3. **Modernizing the existing WordPress install** — option A in the Technical White Paper §3.1 — would have addressed the headers but inherited the operational obligations of the WordPress ecosystem indefinitely.
4. **A headless-CMS-plus-static-frontend rebuild** — the option chosen — would by construction eliminate most of the OWASP Top 10 categories that plague long-running WordPress installs (see §6.4) while giving the org full control over response configuration.

The F-to-A grade transition is therefore not just a numerical improvement on a single scanner; it is a downstream consequence of an architectural decision. Cloudflare Pages exposes the response configuration as a single declarative file (`_headers`) versioned alongside the site's code, where it is reviewable, auditable, and easily updatable by any future Web Director who can read English.

HTTP Security Headers

The first layer of defense is the response itself — what the server tells the browser to do with the page before any code executes. This layer is configured in a single file at the repository root and is what produces the A grade in Figure 1.2.

5.1 The `_headers` File

Cloudflare Pages reads a file named `_headers` at the deploy root and applies its directives to every response. The file is plain text, version-controlled alongside the site's code, and is the single source of truth for the site's response configuration. The Bearcast deployment ships the following file:

```
/*
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000; includeSubDomains
Permissions-Policy: camera=(), microphone=(), geolocation=(),
                    payment=(), usb=(), magnetometer=(),
                    gyroscope=(self), accelerometer=(self),
                    interest-cohort=()
Referrer-Policy: strict-origin-when-cross-origin
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Content-Security-Policy: [see §6]
```

Every directive in this file was a deliberate choice. The sections below walk through each one, explaining what it does, why this specific value was chosen, and the residual risk that remains.

5.2 `Strict-Transport-Security`

Value shipped:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

Decomposed:

- `max-age=31536000` — 365 days, in seconds. The browser will refuse any plain-HTTP request to `bearcastmedia.com` (or any subdomain) for one year after seeing this header, regardless of what the user types

or what links they click. The one-year value follows the recommendation in RFC 6797 and the OWASP HTTP Security Headers cheat sheet.

- `includeSubDomains` — extends the HTTPS-only requirement to every subdomain of `bearcastmedia.com`. This catches subdomains we haven't created yet, preventing a future `blog.bearcastmedia.com` or `shop.bearcastmedia.com` from being accessible over plain HTTP if accidentally provisioned.

Conspicuously absent: the `preload` directive. The `preload` directive submits the domain to a list hardcoded into Chrome, Firefox, Safari, and Edge — meaning those browsers will refuse plain-HTTP requests to the domain even on a brand-new install that has never seen the HSTS header. This is the strongest possible enforcement but is also effectively *irreversible*: removal from the preload list takes months, during which a misconfigured HTTPS endpoint would render the entire site unreachable. For a student organization with multi-year turnover in the Web Director role, the irreversibility cost is too high. The non-preload HSTS configuration achieves 95% of the security benefit with no recovery risk.

OPERATIONAL NOTE

Once HSTS is enabled and a browser has seen it, that browser will refuse plain-HTTP requests to the domain for one year. Do not enable HSTS until you have confirmed HTTPS works on every subdomain you might use. The Maintenance Runbook §06 covers rollback procedures if a TLS issue arises.

5.3 `X-Frame-Options` and `frame-ancestors`

Value shipped:

```
X-Frame-Options: DENY
Content-Security-Policy: ... frame-ancestors 'none' ...
```

Both directives instruct the browser to refuse to render `bearcastmedia.com` inside an `iframe` on another origin — the canonical clickjacking mitigation. They are shipped together because:

- `X-Frame-Options` is the older directive, still respected by every current browser, and is the only directive understood by some legacy clients and some non-browser HTTP clients.
- `frame-ancestors` is the modern CSP directive, supersedes `X-Frame-Options` in browsers that support both, and accepts more nuanced policies (e.g. allow embedding from specific origins).

Shipping both is canonical practice; modern browsers ignore `X-Frame-Options` if `frame-ancestors` is present, and older clients fall back gracefully.

The values chosen (`DENY` / `'none'`) are the strictest available: `bearcastmedia.com` may not be embedded in any `iframe` anywhere. The site has no legitimate use case for being embedded — there is no widget the org distributes — so the strict value imposes zero functional cost.

5.4 X-Content-Type-Options

```
X-Content-Type-Options: nosniff
```

The only valid value for this header. Instructs browsers not to override the `Content-Type` declared by the server by inspecting response bytes. Without it, a response served as `text/plain` but containing bytes that look like HTML or JavaScript may be reinterpreted by the browser — historically a vector for turning a benign-looking file upload into executable code on the same origin.

Bearcast Media has no file-upload surface, so the practical exposure was minimal even before the header was added. But the header has zero cost and removes an entire category of edge cases from consideration. It is shipped because there is no reason not to.

5.5 Referrer-Policy

```
Referrer-Policy: strict-origin-when-cross-origin
```

Controls what the browser sends in the `Referer` header when navigating from `bearcastmedia.com` to an external site. The value chosen sends:

- Full URL (origin + path) on same-origin navigations.
- Origin only (e.g. `https://bearcastmedia.com/`) on cross-origin navigations to HTTPS destinations.
- Nothing on cross-origin navigations from HTTPS to HTTP destinations (the downgrade case).

This is the value recommended by the W3C as the new browser default and balances privacy (external sites cannot see what specific page on `bearcastmedia.com` a visitor came from) with analytics (external sites can still see that the visit came from `bearcastmedia.com` at all). A strictly more private policy (`no-referrer`) is available but breaks legitimate analytics integrations and was not chosen for that reason.

5.6 Permissions-Policy

```
Permissions-Policy: camera=(), microphone=(), geolocation=(),  
payment=(), usb=(), magnetometer=(),  
gyroscope=(self), accelerometer=(self),  
interest-cohort=()
```

Disables a list of powerful browser APIs that Bearcast Media does not use. Each entry follows the pattern `feature=(allowlist)`, where an empty allowlist `()` means "no origin may use this feature, including the current page itself."

The choices, decomposed:

FEATURE	POLICY	RATIONALE
<code>camera</code>	Disabled	Bearcast does not record visitor video.
<code>microphone</code>	Disabled	Despite being a radio station, the public site never records the listener's audio.
<code>geolocation</code>	Disabled	No location-aware features.
<code>payment</code>	Disabled	No e-commerce.
<code>usb</code>	Disabled	No hardware integration.
<code>magnetometer</code>	Disabled	No compass features.
<code>gyroscope</code>	<code>(self)</code>	Allowed for the page itself only, in case a future visualization wants device orientation. Cannot be granted to embedded iframes.
<code>accelerometer</code>	<code>(self)</code>	Same rationale as gyroscope.
<code>interest-cohort</code>	Disabled	Opts the site out of Chrome's FLoC tracking (now legacy, but the directive is kept for historical browsers).

The defense-in-depth value: if a future XSS sneaks past the output encoding and the CSP, an attacker's injected script still cannot quietly access the visitor's camera or microphone. The browser refuses the API call entirely. This is the kind of "what if I missed one" insurance that distinguishes a thoughtful security posture from a checklist one.

5.7 Cross-Origin Isolation Headers

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
```

Two related headers that together establish `bearcastmedia.com` as a *cross-origin isolated* context. The implications are subtle but real:

- **Cross-Origin-Opener-Policy: same-origin** — prevents windows opened by `bearcastmedia.com` (or windows that open `bearcastmedia.com`) from interacting with the page's `window` object across origins. Closes the vector where a malicious popup can read the opener's URL or DOM.

- **Cross-Origin-Resource-Policy: same-origin** — prevents other sites from loading bearcastmedia.com's resources (e.g. via an `<img>` tag) and using the response timing or rendering behavior for cross-site information disclosure. Mitigates Spectre-class side-channel attacks.

Both headers are recent additions to the browser security toolkit (post-Spectre, ~2018) and are not yet universally deployed across the web. Bearcast ships them because they have no functional cost (the site has no use case for being embedded as a resource elsewhere) and they future-proof the site against attack categories that primarily affect sites serving authenticated content — which Bearcast does not, but might in some future feature.

5.8 Headers Deliberately Not Set

Equally important is what the `_headers` file does not contain. A few items that appear in other security guides were deliberately omitted:

- **Expect-CT** — deprecated as of 2023, with browsers planning to ignore it. Including it would be cargo-culting.
- **Public-Key-Pins** (HPKP) — formally deprecated. Configuration mistakes here have repeatedly caused multi-month outages on sites that adopted it; the consensus is "don't."
- **X-XSS-Protection** — deprecated. Modern browsers no longer respect it, and Chrome explicitly removed support because the feature itself was sometimes exploited by attackers.
- **Feature-Policy** (the predecessor to **Permissions-Policy**) — replaced and unnecessary.

Knowing what *not* to ship is part of a real security posture. A misconfigured deprecated header is a worse outcome than no header at all.

Content Security Policy

Of all the headers in `_headers`, the Content Security Policy is the most consequential. It is also the most likely to cause functional breakage when misconfigured, and the most rewarding when configured correctly. This chapter walks through the entire policy directive by directive.

6.1 The Full Policy

The CSP shipped by Bearcast Media:

```
default-src 'self';
script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
font-src 'self' https://fonts.gstatic.com data:;
img-src 'self' data: https: blob:;
media-src 'self' blob: https://*.radio.co;
connect-src 'self' https://*.apicdn.sanity.io https://cdn.sanity.io
            https://public.radio.co https://*.radio.co;
frame-src https://www.youtube.com https://www.youtube-nocookie.com;
frame-ancestors 'none';
base-uri 'self';
form-action 'self';
object-src 'none';
upgrade-insecure-requests
```

6.2 The Architectural Insight

The CSP's `connect-src` directive is worth examining closely because it reveals a deeper property of the rebuild. It lists exactly four external origins:

- `https://*.apicdn.sanity.io` — Sanity's API CDN for GROQ queries
- `https://cdn.sanity.io` — Sanity's asset CDN for images
- `https://public.radio.co` — Radio.co's metadata API
- `https://*.radio.co` — Radio.co's various subdomains

These are exactly the same four external services that appear in Diagram 1 of the System Architecture Diagram document. The CSP is, in effect, the security manifestation of the architecture itself: every external service the architecture permits has a corresponding entry in the CSP, and the browser refuses to contact anything else.

This is a deeper property than it may first appear. The CSP is not protecting an arbitrary collection of allowlisted hosts — it is enforcing the architectural decision that `bearcastmedia.com` communicates with exactly `Sanity` and `Radio.co` and nothing else. If a future contributor accidentally adds a `fetch('https://analytics.example.com')` call to the JavaScript, the browser will block it at runtime, the DevTools console will display the violation, and the architectural integrity is preserved by the browser's enforcement rather than by reviewer discipline alone.

6.3 Directive-by-Directive Walkthrough

`default-src 'self'`

The fallback for every fetch directive not explicitly named. `'self'` means "the same origin as the document itself" — in Bearcast's case, `https://bearcastmedia.com`. This is the most restrictive sensible floor: every other directive *widens* the policy for a specific resource type, never narrows it.

`script-src 'self' 'unsafe-inline'`

The hardest directive to get right and the one with the largest residual risk. The two values:

- `'self'` — only scripts from `bearcastmedia.com` itself may execute. No CDN scripts, no Google Analytics, no jQuery from a public CDN.
- `'unsafe-inline'` — inline `<script>` tags and inline event handlers (e.g. `onclick="..."`) may execute.

`'unsafe-inline'` is, exactly as its name suggests, unsafe. It is the reason the Security Headers scanner caps the grade at A and does not give A+. The presence of `'unsafe-inline'` means that if an XSS payload succeeds in reaching the DOM, it will execute — the CSP cannot distinguish injected inline scripts from page-author inline scripts.

The reason it ships is operational: the inherited Bearcast codebase uses approximately thirty inline event handlers (`onclick`, `onerror`, etc.). Removing `'unsafe-inline'` would require refactoring every one of those handlers to use `addEventListener` instead — a non-trivial code change. The audit identified this as a known residual risk (see §12.2) and the appropriate next-cycle improvement is the refactor that allows `'unsafe-inline'` to be removed.

RESIDUAL RISK

`'unsafe-inline'` in `script-src` means that if a stored XSS payload reached the DOM despite the output encoding (§07) and schema validation (§08), it would execute. The output encoding and schema validation are the actual defenses against XSS; the CSP here is providing URL-allowlisting defense (the attacker cannot load external scripts) but not inline-execution defense.

```
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com
```

Allows stylesheets from bearcastmedia.com itself, inline `style` attributes, and Google Fonts. The Google Fonts allowance is necessary because the site uses Inter and Crimson Text from Google's font CDN; `'unsafe-inline'` for styles is significantly lower risk than for scripts (the worst an inline style can do is visual defacement) and was accepted as a reasonable tradeoff.

```
font-src 'self' https://fonts.gstatic.com data:
```

Allows fonts from same-origin, Google Fonts' static asset domain, and inline base64-encoded font data. The `data:` scheme is included because some browser fallback paths embed fonts as data URLs; removing it produces spurious console warnings without security benefit.

```
img-src 'self' data: https: blob:
```

Permissive by design. Images can come from same-origin, inline data URLs, any HTTPS source, and blob URLs (used for dynamically-generated images). The `https:` wildcard is required because Sanity's image CDN uses dynamically-assigned subdomains that cannot be enumerated in advance, and because article authors may legitimately embed images from external sources (news photos, album art).

Images are a low-risk content type — they cannot execute code — so the permissive policy here has minimal security cost. The principle in CSP design is to be restrictive where restriction provides safety value and permissive where it does not.

```
media-src 'self' blob: https://*.radio.co
```

Limits audio and video sources to same-origin, blob URLs, and Radio.co's subdomains. This is what permits the embedded radio player to function while blocking any unauthorized audio sources. If a future XSS attempted to inject a malicious audio stream pointing to an attacker-controlled origin, the browser would refuse to load it.

```
connect-src 'self' https://*.apicdn.sanity.io ...
```

The architectural-fidelity directive discussed in §6.2. Limits `fetch()`, `XMLHttpRequest`, `WebSocket`, and `EventSource` targets to exactly the origins the architecture permits. The most important defensive directive in the policy after `script-src`, because it directly prevents data exfiltration — an attacker who managed to execute code on the page still could not send the exfiltrated data anywhere other than the listed origins.

```
frame-src https://www.youtube.com https://www.youtube-nocookie.com
```

Bearcast embeds YouTube videos for BTV episode playback. The `youtube-nocookie.com` domain is YouTube's privacy-preserving embed mode that does not set tracking cookies on the visitor's browser. Both domains are needed because YouTube redirects between them in some contexts. Every other `iframe` source is blocked.

`frame-ancestors 'none'`

The CSP equivalent of `X-Frame-Options: DENY`, discussed in §5.3. Both ship together for the reasons documented there.

`base-uri 'self'`

Restricts what URLs the page can declare via `<base>` elements to the page's own origin. Mitigates a subtle attack where injected HTML uses `<base href="...">` to redirect all relative URLs on the page to an attacker-controlled origin — turning every link, image, and fetch on the page into an attacker resource.

`form-action 'self'`

Restricts `<form action="...">` submission targets to same-origin. Mitigates phishing-via-injection where an attacker injects a form pointing to their own origin to capture submitted credentials.

`object-src 'none'`

Blocks `<object>`, `<embed>`, and `<applet>` elements entirely. These are legacy elements primarily associated with Flash and Java applets, both of which are deprecated and historically associated with sandbox-escape exploits. Bearcast has no legitimate need for any of them. The `'none'` value is the strictest and is the recommended default in current CSP guidance.

`upgrade-insecure-requests`

Instructs the browser to automatically rewrite any `http://` URL in the page to `https://` before fetching. This is a transitional directive — in a properly-configured site there should be no such URLs — but its presence catches the case where a future contributor accidentally hardcodes a plain-HTTP URL. The directive is cheap (zero functional cost) and provides a margin of safety.

6.4 OWASP Top 10 Mapping

The Open Web Application Security Project's "Top 10" list is the standard reference for web-application risk categories. The 2021 edition (most recent at time of writing) enumerates ten categories. The combination of the architectural choices (Technical White Paper §11.2) and the CSP/header configuration documented in this chapter collapses most of these for Bearcast Media:

OWASP CATEGORY	BEARCAST STATUS	MITIGATION
A01 Broken Access Control	N/A (public site)	No authenticated surface on the public site; Studio access via Sanity's identity layer.
A02 Cryptographic Failures	Mitigated	TLS terminated by Cloudflare, HSTS enforced, modern cipher suites by default.
A03 Injection	Mitigated	No SQL surface (no database the site queries). GROQ queries parameterized via Sanity SDK. Output encoding for stored XSS (see §07).
A04 Insecure Design	Out of scope	Bears on Sanity's platform design, not Bearcast's application code.
A05 Security Misconfiguration	Mitigated	No server to misconfigure. Configuration is the <code>_headers</code> file documented here.
A06 Vulnerable Components	Mitigated	Public site has zero npm dependencies. Studio's dependencies managed by Sanity's release cadence.
A07 Authentication Failures	N/A / Vendor	No first-party authentication. Studio auth handled by Sanity.
A08 Software & Data Integrity	Mitigated	Deploys are immutable per-commit; rollback in one click. CSP enforces origin allowlisting.
A09 Logging & Monitoring	Mitigated	Cloudflare edge analytics provide request-level observability without PII collection.
A10 Server-Side Request Forgery	N/A	No server-side request surface on the public site.

Of ten OWASP categories, seven are either fully mitigated or structurally inapplicable to Bearcast's architecture. The remaining three (A04, A07, A10) relate to systems Bearcast does not operate (Sanity, Cloudflare). This is the dividend of the architectural choice to use vendor-managed identity and infrastructure rather than self-hosted — entire risk categories disappear because they are someone else's responsibility.

Frontend Output Encoding

The HTTP headers and CSP from §05 and §06 are essential, but they are not the primary XSS defense — they are the last-line-of-defense seatbelt. The primary defense is correct output encoding at every point where CMS data becomes HTML. This chapter documents the five sanitization helpers built during Phase 1 of the audit and the fourteen sites where they are applied.

7.1 The Five Sanitization Helpers

Phase 1 of the audit found fourteen distinct stored-XSS sinks in `scripts.js`. Rather than patching each individually with bespoke escaping logic — which would inevitably miss cases and accumulate bugs over time — the fix centralized escaping in five global helpers, defined once near the top of `scripts.js` and called from every rendering site that consumes CMS data.

The helpers, in summary:

HELPER	PURPOSE	USED AT
<code>bcEscHtml(s)</code>	HTML-entity escape, including <code>'</code> and <code>`</code>	Any field interpolated into element text or attribute values
<code>bcSafeUrl(u)</code>	Returns <code>about:blank</code> for <code>javascript:</code> , <code>data:</code> , <code>vbscript:</code> schemes; otherwise passes through	Any field interpolated into <code>href</code> or <code>src</code> attributes
<code>bcSafeColor(c, fb)</code>	Allows only hex, <code>rgb()</code> , <code>hsl()</code> , CSS named colors, and <code>var(...)</code> references; otherwise returns fallback	Any CMS-provided color injected into inline <code>style</code> attributes
<code>bcSafeId(s, fb)</code>	Returns string matching <code>[a-zA-Z0-9_-]{1,64}</code> only; otherwise returns fallback	Any field used to construct a CSS class name (e.g. <code>verdict-\${value}</code>)
<code>bcSanitizeHtml(h)</code>	Allowlist-tag HTML sanitizer using inert document parsing; strips <code><script></code> , <code>on*</code> attributes, <code>javascript:</code> URLs, and any unknown tags	The rich-text article body field, which legitimately contains HTML

7.2 The Encoding Strategy

There is a hierarchy of safety in DOM manipulation, and the fix consistently picked the safest available option for each use case:

1. **Prefer `textContent` over `innerHTML`.** When the rendered output is plain text (a name, a date, a numeric score), assign via `textContent`. The browser cannot interpret `textContent` as HTML, full stop. This is the safest option and is used wherever it suffices.
2. **Where templating is needed, escape every variable.** When constructing HTML from a template literal, every `${x}` interpolation passes through `bcEscHtml` at minimum. URL fields additionally pass through `bcSafeUrl`; color fields through `bcSafeColor`; CSS-class identifiers through `bcSafeId`.
3. **For rich content that legitimately is HTML, sanitize with an allowlist.** The article body is a Portable Text rich-text field that renders to genuine HTML — paragraphs, headings, emphasized text, links. Rather than escape this (which would defeat the feature), it passes through `bcSanitizeHtml`, which strips any tag, attribute, or URL scheme not on a hardcoded allowlist.

7.3 The Sanitizer Implementation

The most consequential of the five helpers is `bcSanitizeHtml`. The simpler escaping helpers are standard one-liners; the HTML sanitizer needed enough thought to warrant a dedicated review. Its implementation:

```

// Allowlist of safe tags and their permitted attributes
const ALLOWED_TAGS = {
  p: [], h2: [], h3: [], h4: [],
  strong: [], em: [], u: [], s: [], code: [],
  ul: [], ol: [], li: [],
  blockquote: [], cite: [],
  br: [], hr: [],
  a: ['href', 'rel', 'target'],
  img: ['src', 'alt'],
  figure: [], figcaption: [],
};

function bcSanitizeHtml(html) {
  // Parse in an inert document – no script execution, no resource loading
  const doc = document.implementation.createHTMLDocument('');
  doc.body.innerHTML = String(html || '');

  const remove = [];
  for (const node of doc.body.querySelectorAll('*')) {
    const tag = node.tagName.toLowerCase();
    if (!(tag in ALLOWED_TAGS)) {
      remove.push(node);
      continue;
    }
    const allowed = ALLOWED_TAGS[tag];
    for (const attr of [...node.attributes]) {
      const name = attr.name.toLowerCase();
      if (!allowed.includes(name)) {
        node.removeAttribute(attr.name);
        continue;
      }
      // href/src: only safe URL schemes
      if (name === 'href' || name === 'src') {
        node.setAttribute(name, bcSafeUrl(attr.value));
      }
    }
    // Force noopener/noreferrer on every external link
    if (tag === 'a') {
      const rel = (node.getAttribute('rel') || '').toLowerCase();
      if (!rel.includes('noopener')) {
        node.setAttribute('rel', (rel + ' noopener noreferrer').trim());
      }
    }
  }
  // Replace disallowed nodes with their text content
  for (const node of remove) {
    const text = doc.createTextNode(node.textContent || '');
    node.parentNode && node.parentNode.replaceChild(text, node);
  }
  return doc.body.innerHTML;
}

```

Two design choices worth noting:

Inert document parsing. The sanitizer parses the input via `document.implementation.createHTMLDocument('')`, which creates a new HTML document that is not connected to the main document, does not execute scripts, does not load external resources, and does not trigger network requests. This is the canonical browser-native way to parse untrusted HTML — it leverages the browser's own (heavily-audited) HTML parser without any of the live-execution side effects.

Disallowed nodes are replaced with their text content, not deleted. If an editor accidentally writes `<blink>urgent news</blink>`, the result is the text "urgent news," not nothing. This preserves the editor's content while stripping the unsafe markup. Quieter graceful degradation produces fewer "where did my content go?" support tickets.

WHY NOT DOMPURIFY?

[DOMPurify](#) is the industry-standard JavaScript HTML sanitizer, maintained by cure53, and is what most production sites use for this purpose. The Bearcast audit deliberately chose the custom sanitizer above for two reasons: (a) the public site ships zero npm dependencies as an architectural property (Technical White Paper §3.2), and adding DOMPurify would break that property; (b) the threat surface is small enough that a 40-line audited sanitizer is sufficient.

For a higher-stakes deployment, swapping to DOMPurify is the correct upgrade path and is documented in §12.3. The decision boundary is essentially the trust placed in the CMS editor pool: if Bearcast ever opens content creation to untrusted writers, DOMPurify becomes the right answer.

7.4 The Inline-Handler Refactor

Finding XSS-12 in §11 documents a specific class of issue: the journalism page's side-stories used an inline `onclick` handler that interpolated the article ID directly into the JavaScript context:

```
// Before: vulnerable to XSS via the a.id interpolation
sideStoryHtml += `

The fix replaces the inline handler with a data attribute and a single delegated event listener at the container level:



BearcastMedia.com — Cybersecurity White Paper



32 / 68


```



```
// After: data attribute, no JS-context interpolation
sideStoryHtml += `<div class="side-story"
                  data-art-id="${bcEscHtml(a.id)}">
    ...
</div>`;

// Single listener on the container, delegated to children
sideStoriesContainer.addEventListener('click', (e) => {
    const story = e.target.closest('.side-story');
    if (story) {
        openArticle(story.dataset.artId);
    }
});
```

This is a small refactor with a large security implication. Data attributes are always treated as strings; they cannot be "broken out of" the way a JavaScript-context interpolation can. Even if `a.id` contained `1)"><script>alert(1)</script>`, it would be rendered as a literal string in the `data-art-id` attribute, not as executable code.

7.5 The Fourteen XSS Sinks, Categorized

The full enumeration is in §11. Categorized by the kind of field-to-DOM context that produced each:

CATEGORY	COUNT	EXAMPLE	FIX APPLIED
Plain-text field into element text	5	Article title in card	<code>bcEscHtml()</code>
URL field into <code>href</code>	2	External link in article body	<code>bcSafeUrl()</code> + <code>bcEscHtml()</code>
Color field into inline style	3	Show accent color in radio grid	<code>bcSafeColor()</code>
Identifier field into CSS class	1	Review verdict class	<code>bcSafeId()</code> (later combined with schema enum)
Rich HTML into article body	2	Article body modal content	<code>bcSanitizeHtml()</code>
External API response (Radio.co)	1	Currently-playing track display	<code>bcEscHtml()</code>

Every site identified in Phase 1 now passes its data through one or more of these helpers before reaching the DOM. The sanitization helpers are documented at the top of `scripts.js` with security-rationale comments so that future contributors know why they exist and resist the temptation to bypass them.

7.6 The Trust-Boundary Insight

A common misunderstanding about output encoding is that it is only necessary when the input is untrusted. The pre-audit Bearcast codebase implicitly assumed this: the CMS editors are trusted Bearcast directors and contributors, therefore their content does not need encoding. This reasoning is wrong for several independent reasons:

- **Compromise.** An editor's account can be compromised. The instant it is, the trust model is invalidated — but the code that ran under the previous trust model is still running.
- **Future change.** The editor pool can grow. A site that today has three trusted editors might tomorrow accept guest contributors, intern submissions, or reader comments. Without consistent encoding, each expansion of the trust pool requires re-auditing every rendering site.
- **Defense in depth.** Output encoding is essentially free to apply. The cost of applying it is one function call per interpolation; the cost of *not* applying it and being wrong about an input source is total compromise.
- **External APIs.** Bearcast's Radio.co metadata is not from a trusted editor — it is from an external API over a network path that may be hostile. A site that encodes CMS data but not Radio.co data has correctly identified one trust boundary and missed the other.

The principle adopted post-audit: **treat every piece of data as untrusted at every rendering site, regardless of its origin.** This is the discipline that distinguishes a secure codebase from one that happens to currently be safe.

Schema-Level Input Validation

The frontend output encoding from §07 protects every rendering site. But it does so at the latest possible moment — when the data is already in the visitor's browser and about to be displayed. Catching bad data at the input layer, in the Sanity Studio before the content is ever published, is a cleaner and earlier defense. This chapter documents the eight schema-level findings from Phase 2 of the audit and the defense-in-depth pattern they establish.

8.1 Why Input Validation Matters Even With Output Encoding

A frequent objection to layered defenses is "if the frontend encodes correctly, the schema doesn't need to validate." This is wrong in three ways:

- **The frontend can change.** A future contributor may write a new rendering site that forgets to call the sanitization helpers. Without input validation, the bad data is already in the database, waiting to be rendered unsafely by the new code.
- **Bad data is operationally expensive even if it doesn't execute.** A megabyte-long article title pasted by an editor will not produce XSS — but it will produce broken layouts, slow GROQ queries, and inflated API responses. Validation catches these before they become support tickets.
- **Bad data should be rejected at the earliest point.** The closer to the source the rejection happens, the clearer the error message to the person who caused it. An editor who sees "this URL must start with https://" in the Studio gets a learnable correction; a visitor who sees a broken page gets nothing.

8.2 The Eight Schema Findings

Phase 2 of the audit found eight issues across the twelve schema files. Each is documented fully in §11. Categorized by issue type:

R-1: URL Fields Used Sanity's Default Validator

Every URL field (`link.href` in `richBody.ts` , the social media URLs in `siteSettings.ts` , the slides URL in `gbm.ts`) was declared as `type: 'url'` with no `uri()` validator. Sanity's default URL validator

accepts a wide range of schemes including `ftp` and `irc`, and crucially does *not* block `javascript:` unless explicit scheme restrictions are configured.

The fix applied to every URL field in the schema:

```
{
  name: 'href',
  type: 'url',
  title: 'URL',
  // SECURITY: explicitly restrict URL schemes. Sanity's
  // default url type does not block javascript:.
  validation: (r: any) =>
    r.required()
      .uri({
        scheme: ['http', 'https', 'mailto', 'tel'],
        allowRelative: true,
      }),
}
```

This is paired with the frontend's `bcSafeUrl()` helper. Defense-in-depth: even if a future schema migration weakens the input validator, the frontend still catches `javascript:` at render time. And even if the frontend bypass occurs, the schema rejects the value at input. Both layers would have to fail for an exploit to succeed.

RS-1, RV-1: Loosely-Typed Fields Used in Sensitive Contexts

Two fields were free-text strings in the schema but used in contexts where the frontend assumed a constrained format:

- `radioShow.accentColor` — used as a CSS color value in inline `style` attributes. An editor could save `"red; </style><script>..."` and the unfortified frontend would inject it as-is.
- `review.verdict` — used to construct a CSS class name (`verdict-${value}`). An editor could save anything; the frontend would build a class from it. Worse, only four values match real CSS rules (`essential`, `great`, `good`, `mixed`), so any other input produced silently-broken cards.

The fixes applied:

```
// accentColor: hex regex constraint at input
{
  name: 'accentColor',
  type: 'string',
  validation: (r: any) =>
    r.regex(/^#[0-9A-Fa-f]{6}$/, {
      name: 'hex-color',
      invert: false,
    }),
}

// verdict: closed enum, only the four valid values
{
  name: 'verdict',
  type: 'string',
  options: {
    list: [
      {title: 'Essential', value: 'essential'},
      {title: 'Great', value: 'great'},
      {title: 'Good', value: 'good'},
      {title: 'Mixed', value: 'mixed'},
    ],
    layout: 'radio',
  },
}
```

The enum is the more important fix of the two. By replacing a free-text field with a four-option radio button in the Studio, the entire class of "typo causes silently-broken card" bugs disappears. The editor cannot enter an invalid value because the input control does not allow them to.

Length Caps Across All Free-Text Fields

Pre-audit, most text fields in the schema had no `max()` length constraint. An editor could paste a multi-megabyte article body, a 100,000-character review headline, or a malformed copy-paste from a Word document containing embedded image data. None of these produce XSS, but all of them produce one or more of: bloated API responses, slow GROQ queries, broken visual layouts, or — in the worst case — exceeding Sanity's hard 5MB-per-document limit and silently failing publishes.

The fix applied: every free-text field now has an explicit `max()` based on its semantic purpose. Headlines cap at 180 characters; decks at 300; biographies at 2,000; rich-text article bodies at no per-field cap but with a runtime warning at 50KB.

SS-1: Deprecated Singleton Enforcement

The `siteSettings.ts` schema used `__experimental_actions: ['update', 'publish']` to enforce singleton-ness — meaning editors could update and publish the document but not create new ones or delete existing

ones. This is the Sanity v2 pattern. In Sanity v3+ (which Bearcast uses), the pattern is officially deprecated and the documentation recommends moving enforcement to `sanity.config.ts` via the `document.actions` filter — covered in §09.

The deprecated approach still functions today, but its continued operation is at Sanity's discretion. The fix moved enforcement to the supported location and documented the migration with a comment in `siteSettings.ts` so that future contributors do not re-introduce the deprecated pattern.

P-1: Privacy-Misleading Field Label

The `person.email` field was labeled "Email (internal)" with the description "Not shown publicly. For internal contact only." This was misleading: every document in the Sanity production dataset is publicly readable via the `apicdn.sanity.io` endpoint, including field-level data. Any visitor with curl could query the dataset and obtain every editor's email.

The remediation here is partly schema, partly architectural. The label was updated to "⚠️ **PUBLICLY READ-ABLE**" with a description that makes the reality explicit. The deeper fix — actually making the field private — would require moving editor contact information out of Sanity entirely (e.g. to a UC-IT-managed directory), which is outside this paper's scope.

HONEST DISCLOSURE

The Sanity dataset is publicly readable by design. This is the architectural property that allows the public frontend to read content without authentication. It is not a bug. But it does mean that every field in every document is, by default, public — and any field labeled "internal" must either actually be public-safe or must be moved out of Sanity. The P-1 finding is in this paper as an information-disclosure risk; the architectural fix is documented in §12.

T-1: Cross-Field Validation Missing

The `timeSlot.ts` object permits an editor to save a time slot where `endTime === startTime` — a zero-length airing. The frontend's "currently on air" calculator treats this as a slot that is never active, causing legitimate shows to appear absent from the schedule for the duration the editor intended.

The fix applied a cross-field validator that rejects zero-length slots unless they explicitly straddle midnight (the `22:00 → 02:00` case, which the rest of the pipeline handles correctly).

8.3 The Defense-in-Depth Matrix

The final state of input validation across the eight findings, paired with the frontend's catches:

THREAT	SCHEMA (INPUT)	FRONTEND (OUTPUT)
<code>javascript:</code> URLs	<code>uri({scheme: [...]})</code>	<code>bcSafeUrl()</code>
CSS color injection	Hex regex	<code>bcSafeColor()</code>
Invalid CSS class	<code>options.list</code> enum	<code>bcSafeId()</code>
Bad YouTube IDs	11-char alphanumeric regex	Regex on render
HTML in plain-text fields	<code>max()</code> + metachar filter	<code>bcEscHtml()</code>
Megabyte length-bombs	<code>max()</code> caps	(no separate frontend defense)

The pattern is consistent: every threat is caught at both layers. The two-layer redundancy is what makes the post-audit posture resilient to future code changes that might accidentally weaken one layer.

Studio Configuration Hardening

Phase 3 of the audit examined the Studio's infrastructure files — not the content schemas, but the configuration that governs how the Studio itself behaves. Six findings were identified, most subtle, the most consequential being incomplete singleton enforcement that allowed editors to accidentally create duplicate site-settings documents.

9.1 Scope

Three files were in scope:

- `sanity.cli.ts` — the Studio's CLI configuration, used by `npm run sanity deploy` and related commands.
- `sanity.config.ts` — the Studio's runtime configuration, including plugins, document-action filters, and the new-document-options filter.
- `structure.ts` — the custom desk structure that produces the department-grouped Studio navigation described in the Editor Handbook.

9.2 C-1: Incomplete Singleton Enforcement (High Severity)

The most important finding of Phase 3. The Studio is supposed to enforce that exactly one `siteSettings` document exists in the dataset — there is no scenario where two site-settings documents make sense. The frontend GROQ query `*[_type == "siteSettings"][0]` returns whichever document Sanity happens to index first if there are multiple; the result is essentially nondeterministic.

The pre-audit code attempted singleton enforcement at one point:


```
// Pre-audit: filter the document action menu
document: {
  actions: (prev, {schemaType}) => {
    if (schemaType === 'siteSettings') {
      return prev.filter(({action}) =>
        !['delete', 'duplicate'].includes(action)
      );
    }
    return prev;
  },
}
```

This filters the menu shown *on an open document*: an editor with siteSettings open cannot click Delete or Duplicate. But it does not prevent the editor from clicking the global "+" button in the Studio's top nav and selecting "New Site Settings" — creating a brand-new (and now second) document of that type.

The fix added a second filter at a different layer:

```
// Post-audit: also block new-document creation via global +
document: {
  newDocumentOptions: (prev, {creationContext}) => {
    // Filter out siteSettings from the global new-doc menu
    return prev.filter(({templateId}) =>
      templateId !== 'siteSettings'
    );
  },
  actions: (prev, {schemaType}) => {
    if (schemaType === 'siteSettings') {
      return prev.filter(({action}) =>
        !['delete', 'duplicate', 'unpublish'].includes(action)
      );
    }
    return prev;
  },
}
```

The fix also extends the open-document filter to include `unpublish` — which the original code missed. An editor could previously unpublish the singleton (making the public site lose its site-settings entirely until a republish), which is functionally indistinguishable from a delete from the visitor's perspective.

Verification: after the fix was applied, the global "New document" menu no longer offers Site Settings as an option, and the open-document menu hides Delete, Duplicate, and Unpublish. The desk structure (`structure.ts`) opens the singleton directly, so editors reach it from a different path entirely.

9.3 CLI-1: Missing `autoUpdates` Configuration

The CLI configuration did not specify `deployment.autoUpdates`, meaning the deployed Studio at `bear-cast.sanity.studio` kept running whatever Sanity Studio version was current at the time of the last deploy. Sanity ships updates frequently, some of which include security fixes; without auto-updates, the deployed Studio could be running months-old code.

The fix added the directive at the correct location for the v3+ syntax:

```
defineCliConfig({
  api: {
    projectId: '3getuuyu',
    dataset: 'production',
  },
  // FIX: enable auto-updates so the deployed Studio
  // stays current with Sanity's security patches
  deployment: {
    autoUpdates: true,
  },
})
```

A note on the location: Sanity v2 placed `autoUpdates` at the top level of the CLI config. Sanity v3 moved it under `deployment`. Putting it at the wrong location fails silently — the directive is ignored without error.

Verification consisted of confirming that the deployed Studio in production began auto-updating after the fix landed.

9.4 S-1, S-2: Desk Structure Filter Errors

Two related issues in `structure.ts` — the custom navigation that organizes the Studio by department:

- **S-1:** The "Executive Leadership" desk filter was `isExecutive == true`. But the schema only exposes the `isExecutive` field when `isDirector` is true — meaning the filter could in principle match documents where the field was leftover from a person who had been demoted. The fix changed the filter to `isDirector == true && isExecutive == true`.
- **S-2:** The Journalism section had direct navigation entries for only three of the seven article desks (News, Opinion, Investigation). The remaining four (Campus Life, Culture, Science & Tech, Politics) had no direct nav entry, forcing editors to hunt through "All Articles" with manual filtering. The fix added direct entries for every desk.

Neither is a security issue per se; both are operational correctness issues that surfaced during the security audit because reviewing the desk structure required reading the same filter expressions one would read to look for permission bypasses.

9.5 The Configuration as an Architectural Boundary

The Studio configuration is interesting from a security perspective because it is the boundary between *what editors can do* (governed by Sanity's permission system) and *what editors are presented with* (governed by the Bearcast Studio configuration). Even an editor with full delete permissions cannot delete the site-settings singleton if the Studio configuration removes Delete from the action menu — because the UI affordance to do so does not exist.

This is not "true" security — a determined editor could use the Sanity CLI directly to bypass the Studio entirely and perform any operation their token permits. But it is *guardrail* security: it prevents the casual, accidental mistake (the editor who clicks the wrong button while exploring the Studio) from cascading into a site-breaking outcome. For a multi-editor environment with rotating contributors, guardrails matter more than absolute permissions.

Penetration Testing the Defenses

A security audit that ends with "all findings fixed" without independent validation is just a list of code changes. This chapter documents how each of the fourteen XSS findings was pen-tested — first against the pre-fix codebase to confirm the vulnerability was real, then against the post-fix codebase to confirm the fix worked. The methodology is reproducible: any future Web Director can repeat this entire test suite to verify that a change has not regressed any of the defenses.

10.1 Pen-Test Setup

The pen-tests in this chapter exploit *stored* XSS — vulnerabilities triggered by injecting malicious data into the Sanity dataset and then loading the page that renders that data. None of them are reflected XSS; there is no URL that triggers them. The test workflow:

1. **Stand up the pre-fix codebase locally.** Copy the original `scripts.js` and HTML files to a working directory. Run `python3 -m http.server 8000` from that directory. Point a browser at `localhost:8000`.
2. **Log into Sanity Studio** with editor credentials.
3. **Inject the test payload** into the appropriate field of the appropriate document type.
4. **Save and publish** the document.
5. **Reload the affected page** in the localhost browser.
6. **Observe execution** via the DevTools console.

10.2 Test Payload Selection

Every test in this chapter uses the same harmless payload shape:

CANONICAL TEST PAYLOAD

```
<img src=x onerror="console.log('XSS-NN fired'); document.title='PWNED'">
```

This payload is deliberately chosen for three reasons:

- **It is harmless.** Logging to the console and changing the document title leaves no persistent damage. The page can be reloaded and the test artifacts are gone.
- **It produces visible evidence.** The console message proves execution, and the title change is visible in the browser tab — useful when testing against complex pages where the console may scroll past.
- **It uses `onerror` rather than `alert()`.** Modern Chrome rate-limits and sometimes silently drops `alert()` inside certain event-handler contexts, which can make a tester think the payload failed when it actually succeeded. `console.log` is reliable.

Real attackers would substitute payloads that exfiltrate cookies, harvest credentials from form fields, or load additional attacker-controlled scripts. The pen-tests in this chapter establish that the *shape* of the vulnerability is exploitable; the choice of payload is a matter of "what does the attacker want to do with the primitive."

ETHICAL NOTE

Every test in this chapter was performed against Cam's own site, against test documents in Sanity that were deleted after verification. No production content was altered, no real visitor was affected, and no real data was exfiltrated. Pen-testing your own site is the right way to build the intuition for spotting these patterns; pen-testing someone else's site without their explicit authorization is a federal crime under the Computer Fraud and Abuse Act and is not endorsed here.

10.3 Test 1 — XSS-02: Radio Schedule Genre Field

The Vulnerability

The pre-fix `scripts.js` rendered the radio schedule on the homepage using a template literal that interpolated the `genre` field directly into a CSS class context:

```
els.list.innerHTML = RADIO_SCHEDULE.slice(0, 6).map((s, i) => `

The attribute interpolation tds-${s.genre} is the vulnerable point: an attacker who controls s.genre can break out of the attribute and inject arbitrary HTML.



BearcastMedia.com — Cybersecurity White Paper



45 / 68


```

The Test

1. Open Sanity Studio → Radio → an existing `radioShow` document.
2. Edit the `genre` field. Set its value to:

PAYLOAD (XSS-02)

```
pop"><img src=x onerror="console.log('XSS-02 fired')">
```

1. Save the document. The Studio accepts the value because the pre-fix schema has no validation on this field.
2. Reload `localhost:8000/index.html`.
3. Open DevTools → Console.

Expected Result (Pre-Fix)

The console displays "`XSS-02 fired`". Inspecting the rendered HTML in DevTools shows that the `<span class="tds-genre tds-pop"><img ...>` structure was injected verbatim — the browser parsed the attacker's broken-out closing quote, terminated the class attribute early, and treated the `<img>` tag as a real element.

Expected Result (Post-Fix)

Two independent defenses now block this attack:

- The schema (Phase 2 fix) constrains `genre` to an enum of valid music-genre strings; the Studio will not accept the payload at all. The editor sees a validation error before publishing.
- If the schema validation were somehow bypassed, the frontend (Phase 1 fix) now passes `s.genre` through `bcSafeId()` before interpolating into the CSS class context, returning a fallback value for any non-conforming input.

Both layers must fail for the attack to succeed. Verification consisted of confirming the schema rejection (test fails at Studio publish) and, separately, of editing the schema to temporarily accept arbitrary strings, then confirming the frontend still rejects the payload at render.

10.4 Test 2 — XSS-04: BTV YouTube ID Input

The Vulnerability

The pre-fix BTV page accepted a YouTube video ID via an in-page input field (used for ad-hoc episode previews during editorial review). The input was interpolated directly into the `iframe src` attribute:

```
iframe.src = `https://www.youtube.com/embed/${inputValue}`;
```

The YouTube ID format is constrained (11 characters of `[A-Za-z0-9_-]`), but the pre-fix code did not validate this. An attacker — including the visitor themselves if they are testing — could enter arbitrary content that becomes part of the URL.

The Test

1. Open `localhost:8000/btv.html`.
2. Locate the YouTube ID input field.
3. Enter the following:

PAYLOAD (XSS-04)

```
abc"><script>console.log('XSS-04 fired')</script><img src=x
```

1. Submit the form (or trigger whatever event the page uses).

Expected Result (Pre-Fix)

The injected content becomes part of the iframe's `src` attribute, breaking out of the URL context. Whether the script executes depends on the browser's iframe handling, but the page's rendered HTML now contains attacker-controlled markup.

This is a special case among the XSS findings: it is reflected rather than stored, and self-inflicted (the attacker is the visitor typing into their own browser). The practical exposure is therefore low. It is listed in the findings because the input handling pattern was wrong on principle, and because a future feature that accepts a YouTube ID from another source (e.g. a URL parameter for sharing) would inherit the same bug.

Expected Result (Post-Fix)

The post-fix code validates the input against the regex `/^[A-Za-z0-9_-]{11}$/` before allowing it near the iframe. The payload is rejected at the input handler; the iframe never sees it.

10.5 Test 3 — XSS-06: Music Schedule Author Field

The Vulnerability

The pre-fix music page rendered author names in the schedule grid using direct template-literal interpolation:

```
html += `<div class="music-author">${author.name}</div>`;
```

The Test

1. Sanity Studio → People → any `person` with `isDirector` on the music desk.
2. Edit the `fullName` field:

PAYLOAD (XSS-06)

```
<img src=x onerror="console.log('XSS-06 fired')">
```

1. Save. Reload `localhost:8000/music.html`.
2. The payload is in plain text in an element-text context — no attribute-context interpolation needed.

Expected Result

Pre-fix: the `<img>` tag is parsed as a real element, the `onerror` fires, console shows "`XSS-06 fired`".

Post-fix: the value passes through `bcEscHtml()`, which renders the literal text

"`<img src=x onerror="...">`" on the page — visible to anyone viewing the music desk grid, ugly, but inert. The editor will notice within minutes and fix the field; the visitor is never at risk.

10.6 Test 4 — XSS-09: Article Body Modal Content (Critical Path)

This is the most consequential of the fourteen findings and deserves the most attention. It is the canonical "give an editor an HTML body field and they will eventually paste something dangerous" pattern.

The Vulnerability

The pre-fix code rendered article bodies inside a modal using direct `innerHTML` assignment:

```
document.getElementById('modalContent').innerHTML = item.content;
```

The `item.content` value comes from the Sanity rich-text body field, which is legitimately HTML — paragraphs, headings, links, emphasized text. The intent is correct (the field is supposed to render as HTML); the implementation is wrong (no sanitization between the CMS field and the DOM).

The Test

1. Sanity Studio → Journalism → any `article` document.
2. Edit the `body` field. In rich-text mode, switch to "Inspect" or "Edit raw" if the Studio supports it; otherwise, add a normal paragraph and then edit the resulting Portable Text JSON to include a custom block.

3. Inject a Portable Text block whose serialized HTML output is:

PAYLOAD (XSS-09)

```
<p>Normal article text...</p> <img src=x onerror="console.log('XSS-09 fired');  
document.title='PWNED' "> <p>...continues normally.</p>
```

1. Publish the article. Reload `localhost:8000/journalism.html` and click the article card to open the modal.

Expected Result (Pre-Fix)

The `<img onerror>` fires the instant the modal opens. Console shows "`XSS-09 fired`"; the browser tab title changes to "PWNED". An attacker version of this payload would steal cookies, harvest input field values, or load additional attacker-controlled scripts via `fetch()`.

Expected Result (Post-Fix)

The post-fix code routes the body through `bcSanitizeHtml()` before assignment:

```
document.getElementById('modalContent').innerHTML =  
    bcSanitizeHtml(item.content);
```

The sanitizer parses the input in an inert document, walks the DOM, and removes any tag not in its allowlist. The `<img>` tag is permitted, but the `onerror` attribute is not — it gets stripped. The rendered modal shows the article body correctly, the image appears as a broken-image placeholder (because `src=x` is not a valid URL), and no JavaScript executes. The console is silent.

WHY THIS TEST IS THE MOST IMPORTANT

XSS-09 is the test that most clearly demonstrates the value of the defense-in-depth approach. Even with the schema validation in place (which limits the Portable Text shape but cannot prevent dangerous content within legitimate HTML), and even with the CSP in place (which allows `'unsafe-inline'` for scripts and therefore cannot block this payload), the sanitizer at the rendering site catches it. Three layers of defense, each of which is insufficient on its own, combine to block the attack.

10.7 Test 5 — XSS-12: Inline-Handler Injection

The Vulnerability

The pre-fix journalism side-stories rendered using an inline `onclick` handler that interpolated the article ID directly into the JavaScript context:

```
html += `

The interpolation ${a.id} sits inside a JavaScript context inside an HTML attribute. An attacker who controls a.id can inject arbitrary JavaScript that executes when the visitor clicks the side-story.



## The Test



1. Sanity Studio → Journalism → an existing article document.
2. If the schema allowed it (pre-fix), set the article's _id to:



### PAYLOAD (XSS-12)



```
1); console.log('XSS-12 fired'); (
```



1. Save and reload localhost:8000/journalism.html.
2. Click on the side-story that references this article.



## Expected Result (Pre-Fix)



The rendered onClick handler becomes:



```
onClick="openArticle(1); console.log('XSS-12 fired'); ())"
```



The browser parses this as three statements: openArticle(1), console.log(...), and the syntactically-broken (). The first two execute; the third throws a syntax error in the console. The attacker's code has run.



## Expected Result (Post-Fix)



The post-fix code uses a data attribute and a delegated event listener:



```
html += `

BearcastMedia.com — Cybersecurity White Paper

50 / 68


```


```

The article ID is now a data attribute (treated by the browser as a literal string, never executed as JS) and is additionally HTML-encoded by `bcEscHtml`. There is no JavaScript-context interpolation anywhere; the attack surface is removed by construction.

10.8 Test 6 — Defense-in-Depth: CSP Blocks the Bypass

This test is conceptually different. Suppose, hypothetically, that the audit missed an XSS sink and an attacker successfully injected:

HYPOTHETICAL BYPASS PAYLOAD

```
<script src="https://attacker.example.com/payload.js"></script>
```

The post-fix CSP includes `script-src 'self' 'unsafe-inline'` — which permits inline scripts (due to the `'unsafe-inline'` concession discussed in §6.3) but forbids scripts loaded from external URLs.

The Test

1. Manually inject a `<script src="https://example.com/anything.js"></script>` into any rendered HTML via DevTools (this simulates a hypothetical bypassed sink).
2. Observe the browser's response.

Expected Result

The browser refuses to load the external script. The DevTools console displays a CSP violation message:

```
// DevTools console output
Refused to load the script 'https://example.com/anything.js'
because it violates the following Content Security Policy
directive: "script-src 'self' 'unsafe-inline'".
```

The browser has done its job as the last line of defense. Even if every prior layer had been bypassed, the CSP would have prevented the most damaging form of XSS (external script loading) from succeeding. This is the value of layered defense.

10.9 Re-Running the Test Suite

The full pen-test suite should be re-run after any of the following events:

- Adding a new field to a schema document type.

- Adding a new rendering site in `scripts.js`.
- Modifying any of the five sanitization helpers.
- Changing the CSP in the `_headers` file.
- Upgrading the Sanity Studio to a major new version.
- Adding a new external service or integration.

The suite takes approximately 90 minutes for a single tester familiar with the methodology. A future Web Director who has read this paper and has editor credentials can complete the full re-validation in one sitting.

10.10 Summary

Every finding identified in the audit has been pen-tested against both the pre-fix and post-fix codebases. The post-fix codebase resists every payload from this chapter. The defenses are not aspirational — they have been independently verified to function as designed.

The full enumeration of findings, their severities, and their remediations follows in §11.

Findings Catalog

Every finding from the three-phase audit, with severity, description, and the remediation applied. Findings are grouped by audit phase. Each finding's identifier follows the convention used in the original audit reports for cross-referencing.

11.1 Severity Distribution

SEVERITY	FRONTEND (PH. 1)	SCHEMA (PH. 2)	STUDIO (PH. 3)	TOTAL
CRITICAL	0	0	0	0
HIGH	14	2	1	17
MEDIUM	2	4	3	9
LOW	3	2	1	6
INFO	1	0	1	2
Totals	20	8	6	34

Zero critical findings is the correct outcome of the threat model in §02 — every XSS is gated on the Sanity editor trust boundary, which downgrades the worst-case impact from "unauthenticated external attacker" to "authenticated editor with intent to harm or whose account has been compromised." Severities reflect that reality.

11.2 Phase 1 Findings — Frontend

XSS-01 HIGH Article body modal — unsanitized `innerHTML`

The journalism article modal's body content was assigned to `innerHTML` directly from the Sanity rich-text field. Any HTML, including `<script>` tags and `on*` event handlers, would execute on modal open.

LOCATION `scripts.js` · `openArticle()` · modal-content assignment

TRIGGER Any editor publishes an article whose body contains malicious HTML

FIX Route through `bcSanitizeHtml()` (§7.3); allowlist-strip every tag, attribute, and URL scheme not on the safe list

VERIFIED Pen-test §10.6 (XSS-09 in the original report enumeration)

XSS-02 HIGH Radio schedule genre — attribute-context interpolation

The homepage radio schedule interpolated the `radioShow.genre` field into a CSS class context (`class="tds-genre tds-${genre}"`) without escaping. An editor could break out of the attribute with a payload containing `">`.

LOCATION `scripts.js` · radio schedule renderer · line ~926

FIX Schema enum constraint (Phase 2) + `bcSafeId()` at render (Phase 1)

VERIFIED Pen-test §10.3

XSS-03 HIGH Show accent color — CSS injection

The `radioShow.accentColor` field was interpolated into inline `style` attributes without validation. An editor could inject CSS that broke out of the attribute and into the surrounding HTML context, or that used CSS exfiltration techniques to leak data.

FIX Hex regex on input (Phase 2) + `bcSafeColor()` at render (Phase 1)

XSS-04 MEDIUM BTV YouTube ID input — reflected URL injection

The BTV page accepted a YouTube video ID via an in-page form and interpolated it into the iframe `src` attribute without validation. Self-inflicted (the input is the visitor's own), but the input handling pattern was wrong on principle.

FIX Strict regex validation (`^[A-Za-z0-9_-]{11}$`) before iframe assignment

VERIFIED Pen-test §10.4

XSS-05 HIGH Featured show card — author name interpolation

Author names rendered in featured-show cards were interpolated into element text via template literal without escaping. Plain text-context interpolation, so the simplest `<img onerror>` payload works.

FIX `bcEscHtml()` at every interpolation point

XSS-06 HIGH Music schedule author — element-text interpolation

Same pattern as XSS-05, on the music page. Same fix.

VERIFIED Pen-test §10.5

XSS-07 HIGH Music review card — multi-field unsafe rendering

Music review cards rendered headline, album, artist, and category via direct template-literal interpolation. The card-rendering function was rewritten to pass every field through `bcEscHtml()` and to validate the accent color via `bcSafeColor()`.

XSS-08 HIGH Music journalism card — same pattern as XSS-07

Same card-rendering pattern, applied to journalism cards within the music page. Same fix.

XSS-09 **HIGH** Music modal body content — unsanitized HTML insertion

The music page's modal opened with body content assigned directly to `innerHTML`. The most consequential of the XSS findings because the rich-text body is where editors most freely paste content.

FIX `bcSanitizeHtml()` with full allowlist enforcement

VERIFIED Pen-test §10.6 (canonical case study)

XSS-10 **HIGH** Sports recap card — multi-field unsafe rendering

Sports recap cards (game coverage and sports journalism) followed the same pattern as the music cards. Same fix.

XSS-11 **HIGH** Sports modal body content — unsanitized HTML insertion

Sports modal followed the same pattern as the music and journalism modals. Same fix via `bcSanitizeHtml()`.

XSS-12 **HIGH** Journalism side-story — inline `onclick` handler injection

The journalism page's side-story tiles used an inline `onclick` handler that interpolated the article ID into the JavaScript context (`onclick="openArticle(${a.id})"`). The most subtle of the findings — the interpolation site sits inside a JavaScript context inside an HTML attribute, requiring careful payload construction but allowing arbitrary code execution when triggered.

FIX Replaced inline handler with `data-art-id` attribute and a single delegated event listener at the container level. No JS-context interpolation remains.

VERIFIED Pen-test §10.7

XSS-13 **HIGH** Journalism modal — same pattern as XSS-09, XSS-11

Same modal pattern, on the journalism page. Same `bcSanitizeHtml()` fix.

XSS-14 HIGH Radio.co metadata — external API response injected

The "Now Playing" widget rendered the track title and artist from Radio.co's `station-info` endpoint by direct `innerHTML` assignment. Unlike the other XSS findings (which require Sanity editor compromise), this one is reachable by any attacker positioned on the network path between Radio.co and the visitor — most plausibly a hostile Wi-Fi network.

TRIGGER Network path attacker (Adversary D) substitutes the Radio.co response with malicious content

FIX `bcEscHtml()` at the rendering site. CSP `connect-src` additionally restricts the API endpoint to `*.radio.co` so the attacker cannot redirect to their own origin.

SEC-15 HIGH No Content Security Policy

The site shipped no CSP header (and no meta-tag fallback). Without CSP, every XSS finding above lacks a last-line-of-defense: even if all output encoding were missed, the browser would have nothing to refuse.

FIX Full CSP shipped in `_headers` and as `<meta>` fallback (§06)

VERIFIED External scan via Security Headers (Figure 1.2)

SEC-16 INFO Public Slack invite link

The Join Us page included a Slack workspace invite link with the `zt-...` shared-invite format. This URL lets anyone with the link join the workspace. On a public site, "anyone with the link" = the entire internet.

STATUS Deliberate choice — the org accepts the tradeoff for recruiting ease. Documented here so the choice is explicit, not accidental.

FUTURE OPTION Replace with a form-based invitation request gated by the directors, if the org's recruitment posture changes.

BUG - 19 MEDIUM Join form submission is a stub — silent failure

The join form's submit handler logged the submission to the console and returned a success message without actually sending the data anywhere. Users believed they were applying to join Bearcast Media; nothing was reaching the org.

SEVERITY RATIONALE Not a security issue per se, but a substantial visitor-trust issue. Listed because it surfaced during the audit.

FIX Stub explicitly marked with `console.warn` visible to anyone testing; honeypot field added; recommendation to wire to a real endpoint (Netlify Forms / serverless function) before next launch.

BUG - 20 MEDIUM Song request form — same pattern as BUG-19

Same stub-submission pattern on the radio page's song request form. Same fix and recommendation.

BUG - 21 LOW Sports modal author role — copy-paste bug from music page

The sports modal defaulted the author role to "Music Writer" — copy-pasted from the music page implementation without updating. Cosmetic, but professionally embarrassing if a real reader caught it.

FIX Default changed to "Sports Writer"

BUG - 22 LOW Homepage duplicate content — Music and Radio cards

The homepage's "Music" and "Radio" department cards both listed music content. The Radio card should have listed live shows, recent broadcasts, or upcoming schedule highlights.

FIX Radio card now lists upcoming shows from the schedule

11.3 Phase 2 Findings — Schema

R-1 **HIGH** URL fields without scheme allowlist

Every `type: 'url'` field across all schemas relied on Sanity's default URL validator, which does not block `javascript:` URLs. Affected fields: `richBody.link.href`, `siteSettings.social.*` (six platforms), `gbm.slidesUrl`, plus several others.

FIX Every URL field now declares `uri({scheme: ['http', 'https', ...]})` with an explicit allowlist

RV-1 **HIGH** Review verdict — free-text where enum needed

The `review.verdict` field was free-text in the schema. The frontend built a CSS class name from it (`verdict-${value}`); only four values matched real CSS rules. Any typo produced a silently-broken card.

FIX Replaced with `options.list` enum of four values (Essential, Great, Good, Mixed) rendered as radio buttons in the Studio

RS-1 **MEDIUM** Radio show accent color — unvalidated string

The `radioShow.accentColor` field was unvalidated string. Editors could save anything; the frontend treated it as CSS color value.

FIX Hex-color regex constraint: `/^[0-9A-Fa-f]{6}$/`

T-1 **MEDIUM** Time slot zero-length not rejected

An editor could save a `timeSlot` where `endTime === startTime`. The "currently on air" calculator treated this as a slot that never matches, causing the show to disappear from the schedule grid.

FIX Cross-field validator rejects zero-length slots unless they explicitly straddle midnight

L-1 MEDIUM Free-text fields without length caps

Almost every `type: 'string'` and `type: 'text'` field across the schemas had no `max()` constraint. An editor could paste megabytes of content, bloating API responses and breaking layouts.

FIX Every free-text field now has a `max()` appropriate to its semantic purpose (headlines 180, decks 300, bios 2000, etc.)

SS-1 MEDIUM Deprecated singleton pattern (`__experimental_actions`)

The Site Settings singleton was enforced via the Sanity v2 `__experimental_actions` directive. This is deprecated in v3+ and could be removed in any future Sanity release.

FIX Enforcement moved to `sanity.config.ts` (see C-1 below) using the supported `document.newDocumentOptions` + `document.actions` filters

P-1 LOW "Internal" email field is publicly readable

The `person.email` field was labeled "Email (internal)" with the description "Not shown publicly. For internal contact only." The label was misleading — the entire Sanity dataset is publicly readable via the apicdn.

PARTIAL FIX Label updated to "⚠ PUBLICLY READABLE" with description making the reality explicit

ARCHITECTURAL FIX Move editor emails out of Sanity entirely (deferred — see §12.5)

A-1 LOW Image alt-text not required on every image field

Not every image field across the schemas required alt text. Affects accessibility primarily; not a security finding.

FIX Alt text now required on every visible image field

11.4 Phase 3 Findings — Studio Configuration

C-1 HIGH Incomplete singleton enforcement

Documented in detail in §9.2. The pre-audit code filtered the open-document action menu but did not block the global "+ New document" path, allowing duplicate `siteSettings` documents to be created accidentally.

CLI-1 MEDIUM Studio auto-updates not configured

Documented in §9.3. Without `deployment.autoUpdates: true`, the deployed Studio runs whichever Sanity version was current at deploy time, missing subsequent security patches.

S-1 MEDIUM Executive Leadership filter incomplete

Documented in §9.4. The filter `isExecutive == true` could match documents where the field was leftover from a demoted person. Fix: compound filter `isDirector == true && isExecutive == true`.

S-2 MEDIUM Desk navigation incomplete

Only three of the seven article desks had direct navigation entries. Fix: all seven now appear.

S-3 LOW Duplicate-looking desk icons

Two desk-list items used visually similar icons, leading to editor confusion. Fix: replaced with semantically distinct icons.

DPT-1 INFO dept document type unused by frontend

The `dept` document type exists in the schema but is not queried by the frontend (which uses a hardcoded department array). Two options: wire the frontend to fetch from Sanity (recommended for single source of truth), or remove `dept` from the schemas. Documented as a future architectural decision rather than a fix.

11.5 Notes on Items Not Fixed

Two items were identified but deliberately not fixed within the audit cycle:

- **SEC-16 (Public Slack invite).** The current behavior — an open invite link on the Join page — is the org's deliberate choice for recruitment accessibility. The cost of replacing it with a moderated invitation flow exceeds the security benefit at the current org scale.
- **DPT-1 (Unused `dept` schema).** Removal would simplify the schema but require frontend changes. Wiring the frontend to fetch from `dept` would change the rendering architecture. The decision is deferred for the next architectural review.

Deferring fixes is part of an honest audit. Logging them explicitly prevents them from being forgotten while acknowledging that not every finding is worth fixing immediately.

Residual Risk & Future Work

No security posture is complete. This chapter documents the known residual risks after the audit, what would trigger a follow-up audit, and what work is recommended for the next development cycle.

12.1 Known Residual Risks

RR-1 MEDIUM CSP includes `'unsafe-inline'` in `script-src`

The Security Headers grade caps at A (not A+) because `'unsafe-inline'` appears in the `script-src` directive. This was a necessary concession because the codebase uses approximately thirty inline event handlers, and the audit was scoped to security fixes rather than architectural refactoring.

Mitigation path: Refactor every inline `onclick` / `onerror` handler to use `addEventListener`. Once complete, remove `'unsafe-inline'` from the CSP. Estimated effort: 4–6 hours of focused refactoring.

RR-2 LOW Custom HTML sanitizer rather than DOMPurify

The `bcSanitizeHtml()` helper is custom-built (~40 lines, audited). DOMPurify is the industry-standard alternative, ~20KB minified, with broader bypass coverage and active maintenance.

Mitigation path: Adopt DOMPurify if the editor pool ever expands to include untrusted contributors, or if the architectural commitment to zero-dependency public site is relaxed. Until then, the custom sanitizer is sufficient for the current threat model.

RR-3 LOW No Subresource Integrity on Google Fonts

Google Fonts is loaded via a standard `<link>` tag without an `integrity="sha384-..."` attribute. If Google's font CDN were ever compromised, an attacker could substitute a malicious stylesheet that the browser would execute as CSS.

Mitigation path: Add SRI hashes. Trivial change; deferred only because the failure mode (Google CDN compromise) is improbable and Google rotates font URLs frequently enough that maintaining SRI hashes is operationally annoying.

RR - 4 **LOW** No client-side rate limiting on form submissions

Once the form-submission stubs (BUG-19, BUG-20) are wired to real endpoints, those endpoints will need rate limiting to resist automated abuse. The honeypot fields added during the audit catch dumb bots but not determined attackers.

Mitigation path: Adopt Cloudflare Turnstile (free, no-friction CAPTCHA) when the forms are wired up.

RR - 5 **LOW** Sanity dataset is publicly readable

The architectural choice that allows the public frontend to fetch content without authentication also exposes every field in every document to the public. This is correct for content fields (article body, headlines, etc.) but means that any field marked "internal" must in fact be public-safe.

Mitigation path: Audit every "internal" field. The `person.email` field (finding P-1) is the known case. Move truly-internal data out of Sanity to a separate UC-managed directory if needed.

12.2 What Would Trigger a Follow-Up Audit

The audit documented in this paper was sufficient for the launch of the current architecture. A follow-up audit is warranted when any of the following occur:

- **User-generated content is introduced.** Article comments, reader submissions, song requests routed directly into Sanity, or any other write path from a non-editor — these invalidate the current trust model and require re-evaluation of every XSS sink.
- **Authentication is added.** A reader login system, a member portal, or any flow that handles credentials demands a new threat model and a new set of defenses (session management, CSRF protection, etc.).
- **A new external service is integrated.** Adding analytics, embedded video from a new vendor, third-party widgets — every new origin in the CSP is a new trust boundary worth examining.
- **Sanity changes its security model.** Sanity has occasionally changed defaults and API behaviors. Major version upgrades warrant a schema re-review.
- **A new Web Director arrives.** The role transition is a natural moment to verify that the documented defenses are still in place and that the new Director understands the rationale behind each.
- **A security advisory is published** for Cloudflare, Sanity, Radio.co, or any of the site's external dependencies.

12.3 Recommended Next-Cycle Work

In rough priority order:

1. **Wire the form stubs to real endpoints.** BUG-19 and BUG-20. Visitors are currently submitting forms that disappear into the void. This is the most user-affecting non-security issue surfaced by the audit.
2. **Refactor inline handlers (RR-1).** Removes `'unsafe-inline'` from CSP and pushes the grade toward A+.
3. **Add SRI hashes for Google Fonts (RR-3).** Trivial; should ship with the next routine deploy.
4. **Resolve the P-1 / RR-5 publicly-readable-email situation.** Either accept it explicitly, move the data out of Sanity, or remove the field.
5. **Annual re-run of the pen-test suite (§10).** Build the habit of yearly verification before drift accumulates.

12.4 Vendor Compromise — What If

The threat model in §02 excludes vendor compromise from the protection target. This section briefly considers what happens if a vendor is compromised anyway, because the architectural choices in the rebuild affect the blast radius.

If Sanity Is Compromised

An attacker with write access to the Bearcast dataset could publish arbitrary content to the site. The defenses in §07 and §08 limit this to "displayed text" rather than "executed code" — even a fully compromised Sanity cannot inject JavaScript into visitors' browsers because the frontend sanitizes everything at render time. The blast radius is defacement, not RCE.

If Cloudflare Is Compromised

Cloudflare serves the static assets (HTML, CSS, JS) and is the TLS-terminating endpoint. A Cloudflare compromise could serve malicious JavaScript directly to visitors, bypassing every Bearcast defense — Cloudflare *is* the frontend. This is outside Bearcast's scope by necessity. Mitigation consists of choosing a reputable vendor and accepting the residual risk.

If Radio.co Is Compromised

The Radio.co player is loaded via iframe from a Radio.co subdomain. A Radio.co compromise could play arbitrary audio to listeners. The site's display remains intact; only the audio stream is affected. The blast radius is contained by the same-origin iframe boundary.

If GitHub Is Compromised

The site's source code lives in a GitHub repository. Cloudflare Pages auto-deploys from main on every push. A GitHub compromise that allowed an attacker to push code to the Bearcast repository would result in malicious code being deployed to production within ~60 seconds.

Mitigation: branch protection on `main`, two-factor authentication on every Web Director account with push access, and the rollback procedure in the Maintenance Runbook §6.3. The latter means that even a successful push can be reverted in under a minute once detected.

12.5 What This Paper Does Not Claim

To close: this paper does not claim that bearcastmedia.com is unhackable. No paper makes that claim about any system, and one that does should not be trusted. What the paper claims is more specific:

- The site's externally-visible security posture has moved from F to A on a recognized public scanner.
- Thirty-one findings across three audit phases have been identified, documented, and remediated.
- Every documented finding has been verified post-fix via the methodology in §03 and (for the XSS findings specifically) via the pen-test suite in §10.
- Defense-in-depth has been applied: every threat is caught at two or more independent layers.
- The threat model and residual risks have been documented honestly, including the items that were deliberately not fixed.

This is what an honest security posture document looks like. It is the position from which future improvements can be planned, and from which a successor Web Director can pick up the work without first having to re-establish what the current state actually is.

Colophon & References

13.1 About This Document

This paper is one of four documents that together constitute the Bearcast Media documentation set:

- **Technical White Paper** — architecture, schema, design decisions, comparison to the inherited WordPress template. The "what was built and why" document.
- **Editor Handbook** — how the editorial team uses Sanity to create and publish content. The "how to use it" document.
- **Maintenance Runbook** — how the Web Director keeps the site alive across role transitions. The "how to operate it" document.
- **Cybersecurity White Paper** (this document) — the security posture, audit findings, and pen-test methodology. The "how it stays safe" document.

All four are versioned together and updated as the site evolves. Anyone responsible for the site should be familiar with the Editor Handbook; the Web Director should be familiar with all four.

13.2 References

Standards and Specifications

- RFC 6797 — HTTP Strict Transport Security (HSTS)
- W3C Content Security Policy Level 3 — current at time of writing
- W3C Referrer Policy — defines `strict-origin-when-cross-origin` and related values
- W3C Permissions Policy — successor to Feature Policy

OWASP Resources

- OWASP Top 10 (2021 edition) — mapped against Bearcast's architecture in §6.4
- OWASP HTTP Security Headers Cheat Sheet — informed the `_headers` directive choices in §5
- OWASP XSS Prevention Cheat Sheet — informed the output-encoding strategy in §7
- OWASP Cross-Site Scripting Filter Evasion Cheat Sheet — used in pen-test payload construction in §10

Vendor Documentation

- Sanity v3+ Schema Validation reference — the `uri({scheme: [...]})` pattern in §8.2
- Sanity Studio Singleton Pattern — current recommended approach in §9.2
- Cloudflare Pages Headers reference — the `_headers` file format in §5.1
- Mozilla MDN Web Docs — the canonical reference for every header and CSP directive cited in this paper

Tooling

- Security Headers (Snyk) — `securityheaders.com` — the public scanner used for Figures 1.1 and 1.2
- Mozilla Observatory — secondary verification tool
- DOMPurify (cure53) — referenced as the industry-standard alternative to the custom sanitizer

13.3 Acknowledgements

The Bearcast Media organization, its directors past and present, and the rotating cohort of student editors whose work this site exists to publish. Specific thanks to the members of the org who tolerated the brief period during which the new build had visible security caveats while the fixes were being applied.

The audit was conducted by Cam Garrison with assistance from Claude (Anthropic) for systematic enumeration and code-review tooling. All findings, fixes, and verifications were reviewed and applied by the author; any errors remaining are the author's.

13.4 Production Notes

Set in Georgia and Courier New, with Crimson Text for display headings. Layout via WeasyPrint from a single HTML source file maintained alongside the site's code. The full source of this document — and the audit's intermediate reports, pen-test payloads, and verification logs — are available to the next Web Director on request.

BearcastMedia.com Cybersecurity White Paper · v1.0 · May 2026
Authored by Cam Garrison · For internal Bearcast Media use and portfolio reference.